Postgresql управление состоянием СУБД

Для студентов ССА 3 курса

Параметры работы СУБД

В работе СУБД PostgreSQL имеется большое количество параметров работы СУБД, но по группам они отличаются на три:

- Уровень SIGHUP
- Уровень User/Backend
- Уровень Postmaster Restart

Отличаются они единственным существенным свойством: временем применения

Файлы настроек

```
a_skl@LAPTOP-ALEX:~$ ls /usr/local/pgsql/data
ls: cannot open directory '/usr/local/pgsql/data': Permission denied
a_skl@LAPTOP-ALEX:~$ sudo ls /usr/local/pgsql/data
[sudo: authenticate] Password:
PG_VERSION  pg_commit_ts  pg_logical    pg_serial     pg_subtrans  pg_xact              postmaster.pid
base        pg_dynshmem   pg_multixact  pg_snapshots  pg_tblspc    postgresql.auto.conf
global      pg_hba.conf   pg_notify     pg_stat       pg_twophase  postgresql.conf
logfile     pg_ident.conf pg_replslot   pg_stat_tmp   pg_wal       postmaster.opts
a_skl@LAPTOP-ALEX:~$
```

И хранятся все эти настройки в двух основных файлах настроек: *postgresql.conf* и *postgresql.auto.conf*

Про время действия – первая группа настроек действует **моментально**, сразу как только мы их ввели серверу, вторая группа, начинает действовать только после **следующего входа этого пользователя** в консоль разработчика, третья же последняя группа действует ***ТОЛЬКО*** после **перезагрузки** сервера в целом. Так что если у вас нет возможности *перезагрузить сервер сегодня* то новые установки вы *сегодня* и не увидите.

Способы редактирования конфигурационных файлов

Файл	Назначение	Приоритет	Кто редактирует?
postgresql.conf	Основной конфиг	Низкий	Админ вручную (через `nano/vim`).
postgresql.auto.conf	Автоматический конфиг. Хранит изменения из SQL.	Высокий	Только через команду `ALTER SYSTEM`.

Обратите внимание, что postgresql.auto.conf не редактируется так сказать руками. А только командами ALTER

Применение инструкций уровня SIGHUP

Представьте: к вам прилетают данные из другой системы (например, 1С или SAP). Нужно за ночь залить 50 ГБ данных, построить по ним индексы и сделать так, чтобы утром база летала.

Вот как выглядит этот единый сценарий шаг за шагом:

Подготовка к загрузке (Уровень SIGHUP)

Проблема: Стандартных 4 МБ памяти на сортировку (work_mem) мало. При загрузке CSV-файлов PostgreSQL будет постоянно писать временные файлы на диск, и загрузка займет 10 часов вместо 30 минут.

Действие: Мы временно увеличиваем память для операций сортировки и хеширования.

Команда вывода SQL-запросов уровня SIGHUP

```
a_skl@LAPTOP-ALEX:~$ sudo -u postgres psql
[sudo: authenticate] Password:
sudo: Authentication failed, try again.
[sudo: authenticate] Password:
psql (18.4)
Type "help" for help.

postgres=# BRGIN;
ERROR:  syntax error at or near "BRGIN"
LINE 1: BRGIN;
        ^~~~

postgres=# BIGIN;
ERROR:  syntax error at or near "BIGIN"
LINE 1: BIGIN;
        ^~~~

postgres=# BEGIN;
BEGIN
postgres=*# SET LOCAL work_mem = '128MB';
SET
postgres=*# cCOMMIT;
ERROR:  syntax error at or near "cCOMMIT"
LINE 1: cCOMMIT;
        ^
postgres=!# COMMIT;
ROLLBACK
postgres=#
```

Обратите внимание, что на слайде представлен не чистый **ALTER SYSTEM SET work_mem = '512MB';**, а несколько на первый взгляд усложненный блок:

BEGIN;
SET LOCAL work_mem = ‘128MB';
COMMIT;

Дело в том, что мои аппаратные возможности не позволяют задавать настолько реальные конфигурации что мы обсуждали это первое, и второе, мне бы не хотелось сохранять введенные настройки поскольку сервер у меня развернут на WSL. Кроме того обратите внимание на мои опечатки ☺

Команда вывода SQL-запросов уровня User/Backend

```
postgres=# ALTER ROLE postgres SET maintenance_work_mem = '1GB';
ALTER ROLE
postgres=# \q
a_skl@LAPTOP-ALEX:~$ sudo -u postgres psql
[sudo: authenticate] Password:
psql (18.4)
Type "help" for help.
```

Еще раз обращаю внимание на свои технические ограничения

Проблема: Данные загружены, но теперь нужно построить индексы для быстрого поиска. Операция CREATE INDEX очень прожорлива. Если мы оставим work_mem большим для всех, обычные пользователи могут уронить сервер утром. Но нам нужно много памяти только сейчас и только для админа.

Действие: Мы задаем специальный лимит памяти для обслуживания только для роли postgres.

-- Это изменение подхватится при следующем входе или через SET

ALTER ROLE postgres SET maintenance_work_mem = '4GB';

Результат: Админ строит индексы молниеносно, используя 4 ГБ RAM, пока остальные пользователи продолжают работать со своими скромными настройками. Безопасно и эффективно.

Вступление в силу новых настроек

```
a_skl@LAPTOP-ALEX:~$ sudo -u postgres psql
[sudo: authenticate] Password:
psql (18.4)
Type "help" for help.

postgres=# SHOW maintenance_work_mem;
 maintenance_work_mem
----------------------
 1GB
(1 row)

postgres=#
```

Возвращение стандартных настроек

```
postgres=# ALTER ROLE postgres RESET maintenance_work_mem;
ALTER ROLE
postgres=# SHOW maintenance_work_mem;
 maintenance_work_mem
----------------------
 1GB
(1 row)

postgres=# \q
a_skl@LAPTOP-ALEX:~$ sudo -u postgres psql
psql (18.4)
Type "help" for help.

postgres=# SHOW maintenance_work_mem;
 maintenance_work_mem
----------------------
 64MB
(1 row)

postgres=#
```

Для возвращения старых настроек иы всегда используем команду ***RESET*** **ALTER ROLE postgres RESET maintenance_work_mem;** ***RESET*** всегда используется для возврата настроек.

Команда вывода SQL-запросов уровня Postmaster Restart

Проблема: База выросла. Данных стало в 2 раза больше. Старого кэша в 128 МБ (shared_buffers) катастрофически не хватает. Утром пользователи начнут жаловаться на тормоза, если мы не выделим базе больше общей памяти.

Действие: Мы меняем фундаментальный параметр — размер разделяемого буфера.

ALTER SYSTEM SET shared_buffers = '8GB';

Результат: Сервер перезапускается, выделяет 8 ГБ под общий кэш. Утром база «греет» данные в памяти, и все отчеты пользователей летают.

Команда ALTER SYSTEM SET shared_buffers

```
ostgres=# ALTER SYSTEM SET shared_buffers = '2GB';
LTER SYSTEM
ostgres=# SELECT pg_reload_conf();
 pg_reload_conf
----------------
 t
(1 row)

ostgres=# SHOW shared_buffers;
 shared_buffers
----------------
 128MB
(1 row)

ostgres=#
```

Момент применения команды уровня Postmaster Restart

```
 shared_buffers
----------------
 128MB
(1 row)

postgres=# SELECT pg_reload_conf();
 pg_reload_conf
----------------
 t
(1 row)

postgres=# SHOW shared_buffers;
 shared_buffers
----------------
 128MB
(1 row)

postgres=# \q
a_skl@LAPTOP-ALEX:~$ sudo systemctl restart postgresql-custom
a_skl@LAPTOP-ALEX:~$ sudo -u postgres psql
psql (18.4)
Type "help" for help.

postgres=# SHOW shared_buffers;
 shared_buffers
----------------
 2GB
(1 row)

postgres=#
```

Момент отмены команды уровня Postmaster Restart

```
postgres=# ALTER SYSTEM RESET shared_buffers;
ALTER SYSTEM
postgres=# \q
a_skl@LAPTOP-ALEX:~$ sudo -u postgres psql
psql (18.4)
Type "help" for help.

postgres=# SHOW shared_buffers;
 shared_buffers
----------------
 2GB
(1 row)

postgres=# \q
a_skl@LAPTOP-ALEX:~$ sudo systemctl restart postgresql-custom
a_skl@LAPTOP-ALEX:~$ sudo -u postgres psql
psql (18.4)
Type "help" for help.

postgres=# SHOW shared_buffers;
 shared_buffers
----------------
 128MB
(1 row)
```